Advanced Deep Learning

Paul Swoboda

HHU Düsseldorf

Summer Semester 2026

Table of Contents

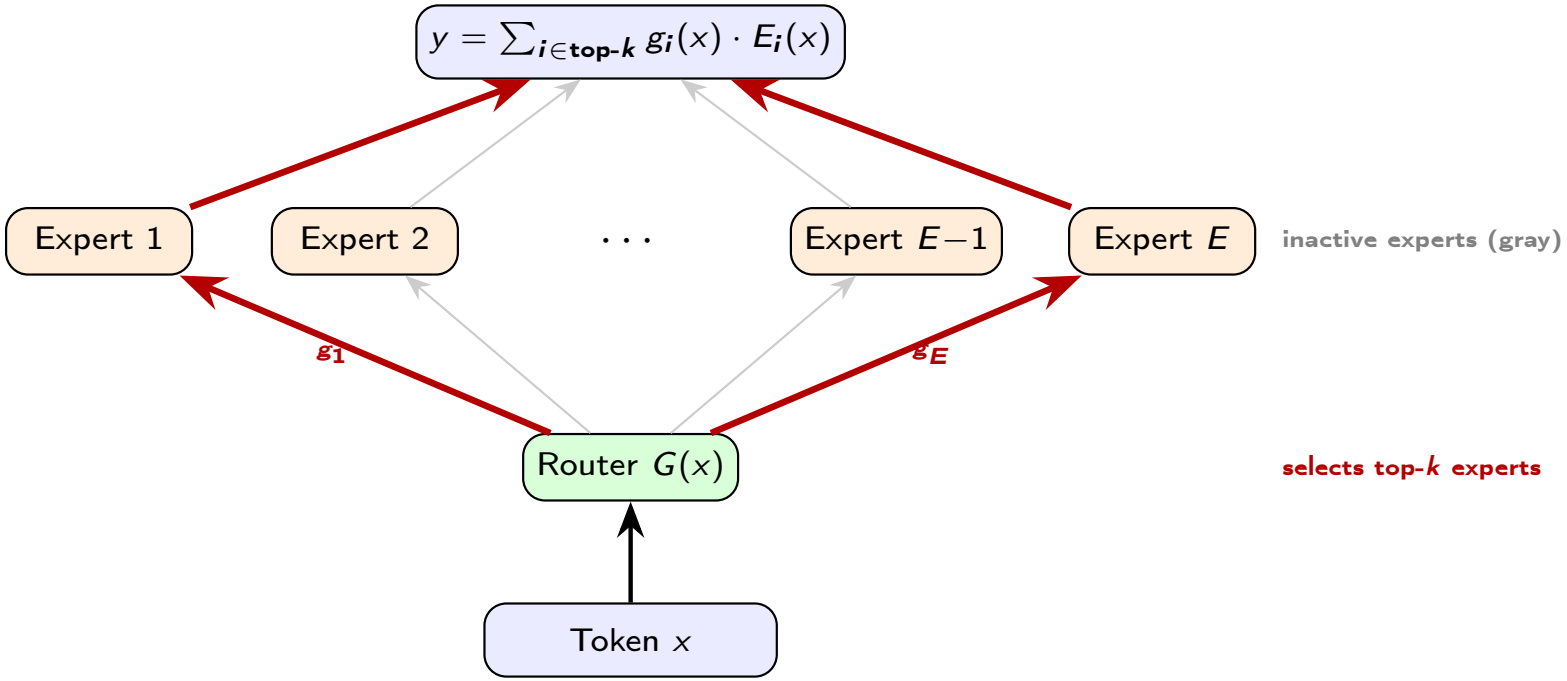
1. Mixture of Experts
   - Motivation and Basic Idea
   - Sparsely-Gated MoE (Shazeer et al., 2017)
   - Switch Transformer
   - MoE in Practice: Integration into Transformers
   - DeepSeek-MoE
   - DeepSeek-V3: MoE at Scale
   - Summary
2. References

Table of Contents

1. Mixture of Experts
   - Motivation and Basic Idea
   - Sparsely-Gated MoE (Shazeer et al., 2017)
   - Switch Transformer
   - MoE in Practice: Integration into Transformers
   - DeepSeek-MoE
   - DeepSeek-V3: MoE at Scale
   - Summary
2. References

MoE: High-Level Idea

Replace the dense FFN in each Transformer block with a **sparse MoE layer**:



Each expert E_i is typically a standard FFN. The **router** (gating network) decides which experts to use per token.

Why Mixture of Experts?

Dense models: every parameter is used for every input token.

- Scaling parameters $\Rightarrow$ scaling FLOPs proportionally
- Training cost grows with model size
- Inference cost grows with model size

Key insight: Not all tokens need the same computation. Can we *decouple* parameter count from compute cost?

Mixture of Experts (MoE)

- Maintain *many* expert sub-networks
- For each token, activate only a *few* experts
- $\Rightarrow$ Large model capacity with sparse computation

Result: Models with N total parameters but only $\sim k/E \cdot N$ active parameters per token (k active out of E experts).

Challenges of MoE

1. **Top-k is not differentiable**
   - The discrete selection of which k experts to activate is a hard, non-differentiable choice
   - Gradients cannot flow through $\mathrm{argtopk}$
   - **Workaround**: Multiply each expert's output by its continuous gating weight $g_i(x)$. Gradients flow through g_i via softmax over the selected experts
   - Router weights W_g are trained by the gating weights, not the selection decision
2. **Load balancing**
   - Router tends to collapse: sends most tokens to a few "favorite" experts
   - Favored experts improve faster $\Rightarrow$ attract even more tokens (self-reinforcing)
   - Requires auxiliary losses or other mechanisms
3. **Communication overhead**
   - Experts distributed across GPUs
   - Tokens must be dispatched to the right device via **all-to-all** communication
   - Poor load balance $\Rightarrow$ stragglers and wasted compute
4. **Memory vs. compute trade-off**
   - All expert weights must reside in memory, even though most are idle per token
   - Total parameter count $\gg$ active parameters
   - Inference requires careful memory management (e.g. expert offloading)
5. **Training instability**
   - Router learning is noisy early in training
   - Requires careful initialization and precision (e.g. router in `float32`)

Table of Contents

1. Mixture of Experts
   - Motivation and Basic Idea
   - Sparsely-Gated MoE (Shazeer et al., 2017)
   - Switch Transformer
   - MoE in Practice: Integration into Transformers
   - DeepSeek-MoE
   - DeepSeek-V3: MoE at Scale
   - Summary
2. References

The Sparsely-Gated Mixture-of-Experts Layer

The seminal work by [Shazeer et al., *"Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer"*, 2017] introduced MoE layers into deep networks at massive scale.

Architecture: Insert an MoE layer between stacked LSTM layers, with up to **131,072 experts** and top-k gating (typically $k = 2$).

Gating network for input x:

$$G(x) = \mathrm{softmax}\Big(\mathrm{TopK}\big(H(x),\, k\big)\Big)$$

where $H(x) = x \cdot W_g + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma^2)$ adds noise for exploration, and

$$\mathrm{TopK}(v, k)_i = \begin{cases} v_i & \text{if } v_i \text{ is in the top } k \\ -\infty & \text{otherwise} \end{cases}$$

MoE output:

$$y = \sum_{i=1}^{E} G(x)_i \cdot E_i(x)$$

Only k terms are non-zero $\Rightarrow$ only k experts are evaluated.

Load Balancing

Problem: Without regularization, the router tends to collapse — sending most tokens to a few "favorite" experts while others remain unused.

Why collapse happens:

- Favored experts get more training signal
- They improve faster ⇒ attract even more tokens
- Self-reinforcing loop

Noisy gating: Adding Gaussian noise ϵ to gating logits helps exploration, but is not sufficient alone.

Auxiliary load-balancing loss:
For a batch of T tokens across E experts, let f_i be the fraction of tokens routed to expert i, and p_i the average router probability for expert i:

$$\mathcal{L}_{\text{bal}} = \alpha \cdot E \cdot \sum_{i=1}^{E} f_i \cdot p_i$$

Minimized when $f_i = p_i = 1/E$ (uniform distribution).

α is a small coefficient (e.g. 10^{-2}) to not dominate the main loss.

Results: Scaling with Sparse MoE

[Shazeer et al., *"Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer"*, 2017] demonstrated on language modeling and machine translation:

- **1000× more parameters** than a dense model at similar computational cost
- Achieved state-of-the-art results on language modeling (1 Billion Word Benchmark) and machine translation
- A model with **137B parameters** (68B active per token with $k = 2$ of 2048 experts) matched or exceeded much larger dense models

Key Takeaway

Sparse MoE enables *conditional computation*: the model can store vastly more knowledge in its parameters without proportionally increasing the compute per example.

Challenges identified:

- Load balancing is critical and non-trivial
- Communication overhead when experts are distributed across devices
- Batch size must be large enough so each expert sees sufficient tokens

Table of Contents

1. Mixture of Experts
   - Motivation and Basic Idea
   - Sparsely-Gated MoE (Shazeer et al., 2017)
   - Switch Transformer
   - MoE in Practice: Integration into Transformers
   - DeepSeek-MoE
   - DeepSeek-V3: MoE at Scale
   - Summary
2. References

Switch Transformer: Simplifying MoE Routing

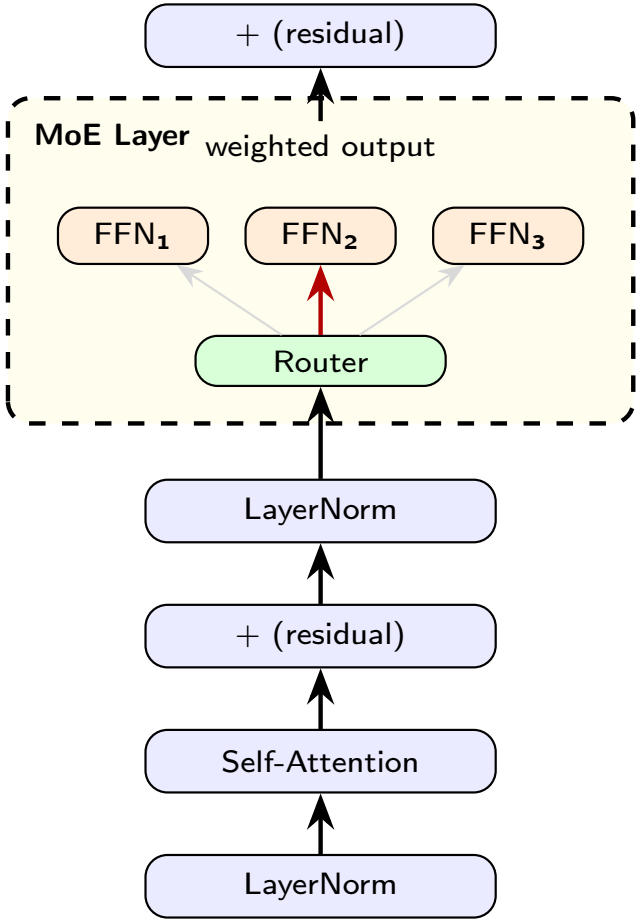
[Fedus et al., *"Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity"*, 2022] simplified the MoE approach with two key changes:

1. **Top-1 routing** ("Switch" routing):
   - Each token goes to *exactly one* expert
   - Simpler than top-k: no need to combine outputs
   - Reduces communication and computation

$$i^* = \arg\max_i (x \cdot W_r)_i$$

$$y = G(x)_{i^*} \cdot E_{i^*}(x)$$

2. **Expert capacity factor**:
   - Each expert processes at most C tokens per batch
   - $C = \frac{k \cdot T}{E} \cdot c$, with capacity factor $c > 1$ (e.g. $c = 1.25$)
   - Overflow tokens are passed through residual connection ("dropped")



Switch Transformer: Results

- Scaled up to **1.6 trillion parameters** (Switch-C) with 2048 experts
- At matched FLOPs, Switch Transformers achieve **4–7**$\times$ **speedup** over dense T5 baselines
- Training instability at large scale mitigated by:
  - Selective use of `bfloat16` (router in `float32`)
  - Smaller parameter initialization
  - Higher expert dropout during fine-tuning

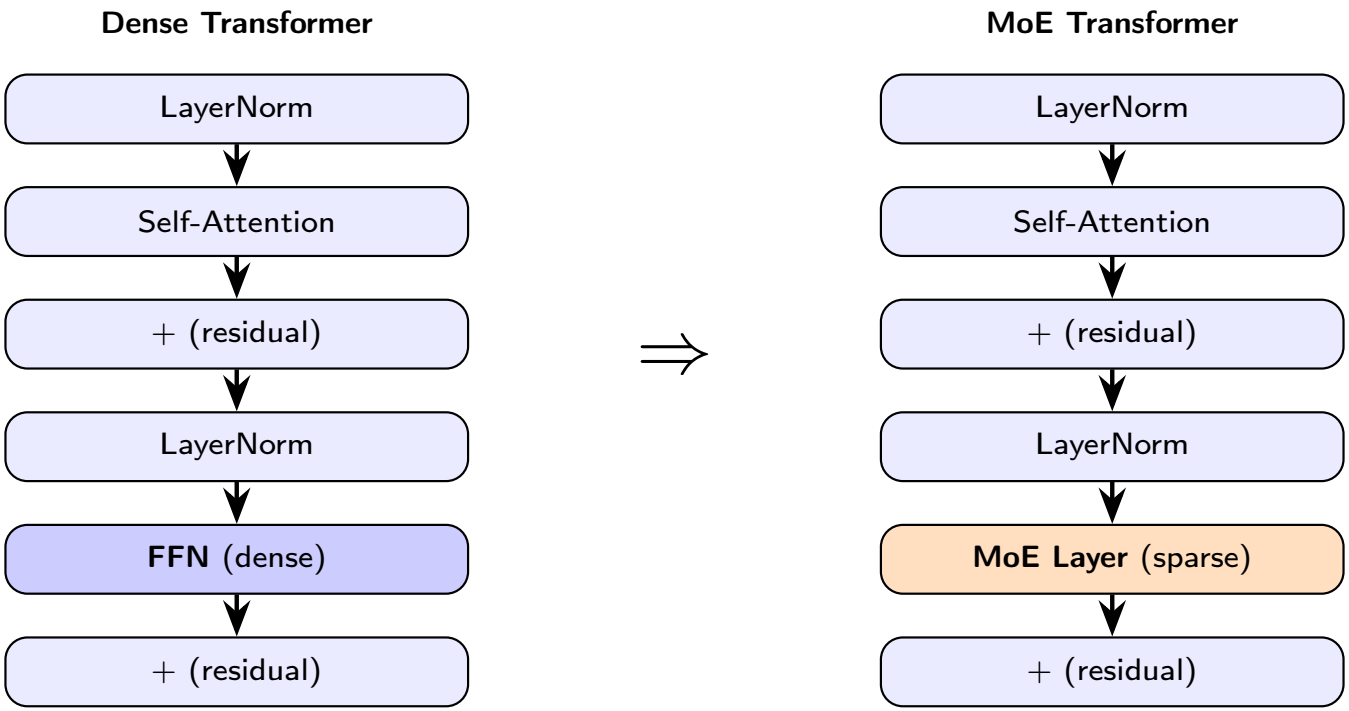
Switch Transformer Design Principles

1. Simplify routing: top-1 is enough
2. Use large but efficient expert count
3. Keep auxiliary loss for load balancing
4. Careful numerical precision for stability

Table of Contents

1. Mixture of Experts
   - Motivation and Basic Idea
   - Sparsely-Gated MoE (Shazeer et al., 2017)
   - Switch Transformer
   - MoE in Practice: Integration into Transformers
   - DeepSeek-MoE
   - DeepSeek-V3: MoE at Scale
   - Summary
2. References

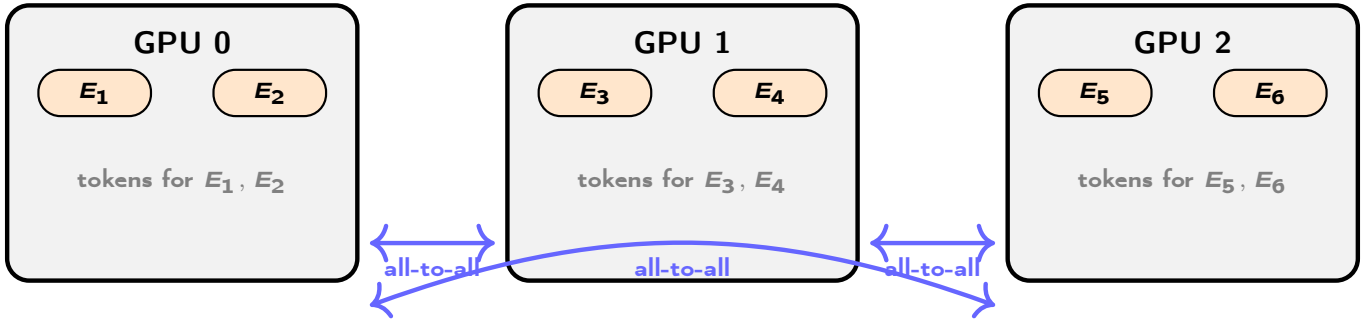
Where Does MoE Fit in a Transformer?



- The **FFN sub-layer** is replaced by the MoE layer (every layer or every n-th layer)
- Attention layers remain **dense** (shared across all tokens)
- Common pattern: alternate dense FFN and MoE layers

Expert Parallelism

When experts are distributed across devices, MoE introduces a new parallelism dimension:



- **Dispatch**: **All-to-all** communication sends each token to the GPU hosting its assigned expert
- **Combine**: After expert computation, **all-to-all** sends results back
- **Cost**: Communication volume depends on load balance — poor balance ⇒ congestion

Table of Contents

1. Mixture of Experts
   - Motivation and Basic Idea
   - Sparsely-Gated MoE (Shazeer et al., 2017)
   - Switch Transformer
   - MoE in Practice: Integration into Transformers
   - DeepSeek-MoE
   - DeepSeek-V3: MoE at Scale
   - Summary
2. References

DeepSeek-MoE: Fine-Grained Expert Specialization

[Dai et al., *"DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models"*, 2024] proposed two innovations to improve expert specialization:

1. **Fine-grained expert segmentation**:
   - Instead of E large experts, split into mE smaller experts (each $1/m$ the size)
   - Activate mk experts (same total compute)
   - Allows more flexible, fine-grained combinations of knowledge

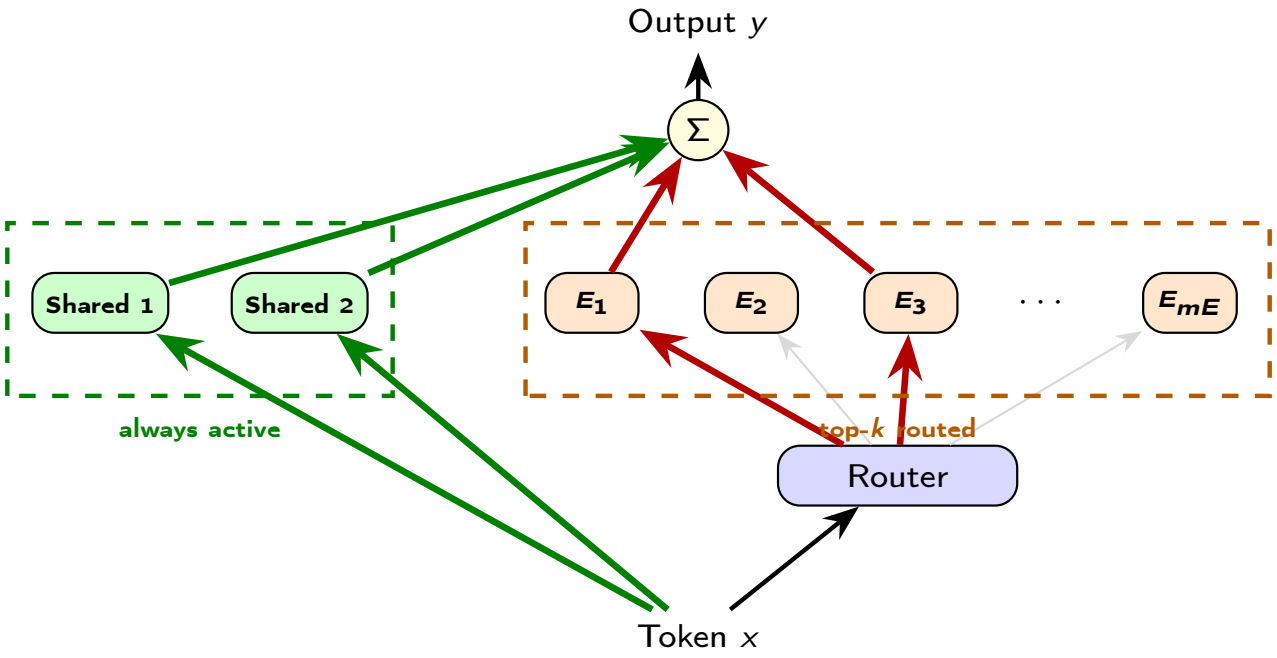
Example: Instead of 16 experts with top-2, use 64 experts with top-8 (same compute, more combinatorial flexibility).

2. **Shared experts**:
   - Dedicate K_s experts as **shared** — always active for every token
   - Remaining experts are **routed** as usual
   - Shared experts capture common knowledge; routed experts specialize

Output

$$y = \sum_{i=1}^{K_s} E_i^{(s)}(x) + \sum_{i \in \text{top-}k} g_i \cdot E_i^{(r)}(x)$$

DeepSeek-MoE: Architecture



- Shared experts reduce *redundancy* among routed experts
- Fine-grained segmentation provides $\binom{mE}{mk}$ possible expert combinations vs. $\binom{E}{k}$ with standard MoE

DeepSeek-MoE: Results

DeepSeek-MoE 16B (2B active parameters, 64 routed experts + 2 shared):

- Comparable performance to LLaMA-2 7B (3.5× more active params)
- Comparable performance to DeepSeek 7B dense model
- ≈40% of the compute of a dense model with equivalent quality

Scaling validation at 145B parameters:

- DeepSeek-MoE 145B approaches the performance of DeepSeek 67B dense
- Only 28.5B active parameters (vs. 67B)
- Confirms that fine-grained + shared experts scale well

Design Recipe

Fine-grained segmentation ($m \approx 4$) + shared experts ($K_s = 2$) consistently outperforms both standard top-k MoE and dense models at matched compute.

Table of Contents

1. Mixture of Experts
   - Motivation and Basic Idea
   - Sparsely-Gated MoE (Shazeer et al., 2017)
   - Switch Transformer
   - MoE in Practice: Integration into Transformers
   - DeepSeek-MoE
   - DeepSeek-V3: MoE at Scale
   - Summary
2. References

DeepSeek-V3: Auxiliary-Loss-Free Load Balancing

[DeepSeek-AI, *"DeepSeek-V3 Technical Report"*, 2024] identified a key problem with auxiliary balancing losses:

Problem

The auxiliary loss $\mathcal{L}_{\text{bal}}$ conflicts with the language modeling objective. Tuning α is difficult: too small $\Rightarrow$ imbalance, too large $\Rightarrow$ hurts model quality.

Solution: Bias-based load balancing (no auxiliary loss):

- Add a learnable **bias** term b_i to each expert's gating score for routing:

$$i^* = \arg\max_i \left(s_i(x) + b_i\right)$$

- The bias is **not** used in the gating weight (only for routing decisions):

$$g_i(x) = \frac{s_i(x)}{\sum_{j \in \text{top-}k} s_j(x)}$$

- b_i is adjusted dynamically: if expert i is overloaded, decrease b_i; if underloaded, increase b_i

$\Rightarrow$ Load balancing is *decoupled* from the training loss.

DeepSeek-V3: Architecture Overview

DeepSeek-V3: 671B total parameters, 37B active per token.

MoE configuration:

- 61 Transformer layers
- First 3 layers: dense FFN
- Layers 4–61: MoE layers
- 256 routed experts + 1 shared expert per MoE layer
- Top-8 routing (out of 256)
- Fine-grained segmentation following DeepSeek-MoE

Additional innovations:

- Multi-head Latent Attention (MLA) for KV cache compression
- Multi-token prediction as auxiliary training objective
- FP8 mixed-precision training

Training efficiency:

- Trained on 14.8T tokens
- Training cost: **2.788M H800 GPU hours**
- ≈$5.5M (remarkably economical)
- Comparable to GPT-4 / LLaMA-3 405B quality

Why so efficient?

- MoE: only 37B of 671B params active
- FP8 training: ∼2× throughput vs. BF16
- Efficient expert parallelism with minimal communication

Comparison: Dense vs. MoE Approaches

Model	Total Params	Active Params	MoE Design
LLaMA-3 405B	405B	405B	Dense
Mixtral 8×7B	47B	13B	Top-2 of 8
Mixtral 8×22B	141B	39B	Top-2 of 8
DeepSeek-V2	236B	21B	Top-6 of 160 + shared
DeepSeek-V3	671B	37B	Top-8 of 256 + shared

Trends in MoE design:

1. **More, smaller experts**: from 8 large → 256 fine-grained
2. **Higher top-k**: from top-1/2 → top-6/8 with smaller experts
3. **Shared experts**: reduce redundancy, capture common knowledge
4. **Better load balancing**: from auxiliary losses → bias-based methods

Table of Contents

1. Mixture of Experts
   - Motivation and Basic Idea
   - Sparsely-Gated MoE (Shazeer et al., 2017)
   - Switch Transformer
   - MoE in Practice: Integration into Transformers
   - DeepSeek-MoE
   - DeepSeek-V3: MoE at Scale
   - Summary
2. References

MoE: Summary and Open Challenges

Key ideas:

- MoE *decouples* model capacity (total params) from compute cost (active params)
- Router selects a sparse subset of experts per token
- Load balancing is essential for training stability and efficiency

Evolution:

- **Shazeer (2017)**: Sparsely-gated MoE layer, noisy top-k gating, auxiliary loss
- **Switch (2022)**: Simplified top-1 routing, scaled to 1.6T params
- **DeepSeek-MoE (2024)**: Fine-grained experts + shared experts
- **DeepSeek-V3 (2024)**: Auxiliary-loss-free balancing, 671B params at low cost

Open challenges:

- **Inference**: All expert weights must be in memory, even though most are idle
- **Expert collapse**: Some experts may remain underutilized
- **Communication**: Expert parallelism requires all-to-all communication
- **Distillation**: Can we compress MoE back into smaller dense models?

Table of Contents

1. Mixture of Experts
   - Motivation and Basic Idea
   - Sparsely-Gated MoE (Shazeer et al., 2017)
   - Switch Transformer
   - MoE in Practice: Integration into Transformers
   - DeepSeek-MoE
   - DeepSeek-V3: MoE at Scale
   - Summary
2. References

References I

Dai, Damai et al. (2024). "DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models". In: *arXiv preprint arXiv:2401.06066*. URL: https://arxiv.org/abs/2401.06066 (cit. on p. 18).

DeepSeek-AI (2024). "DeepSeek-V3 Technical Report". In: *arXiv preprint arXiv:2412.19437*. URL: https://arxiv.org/abs/2412.19437 (cit. on p. 22).

Fedus, William, Barret Zoph, and Noam Shazeer (2022). "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity". In: *Journal of Machine Learning Research* 23.120, pp. 1–39. URL: https://arxiv.org/abs/2101.03961 (cit. on p. 12).

Shazeer, Noam et al. (2017). "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer". In: *International Conference on Learning Representations (ICLR)*. URL: https://arxiv.org/abs/1701.06538 (cit. on pp. 8, 10).